Programación de Servicios y Procesos

Programación multihilo

Tema 2

---

1. Introducción

Programa de flujo único Aquel que realiza las actividades o tarea que lleva a cabo **una a continuación de la otra, de manera secuencial**, lo que significa que cada una de ellas debe concluir por completo antes de iniciar la siguiente.

Programa de flujo múltiple Aquel que coloca las actividades a realizar en diferentes flujos de ejecución que pueden ejecutarse de **manera simultánea o concurrente.**

Programación multihilo Consiste en desarrollar programas o aplicaciones de flujo múltiple. Cada uno de esos flujos es denominado **hilo**.

2. Conceptos sobre hilos

Hilo / subproceso Flujo secuencial independiente dentro de un proceso asociado con una **secuencia de instrucciones, un conjunto de registros y una pila**.

Podemos hacer las siguientes **observaciones**:

- Un hilo **no puede existir independientemente** de un proceso.
- Un hilo **no puede ejecutarse** por si solo.
- Dentro de cada proceso puede haber **varios hilos ejecutándose.**

Ofrecen las siguientes **ventajas**:

- Se consumen menos recursos en el **lanzamiento, ejecución y finalización** de un hilo que de un proceso.
- El cambio de contexto entre hilos del mismo proceso es **más rápido** que entre distintos procesos.

Se **aconseja** usar hilos cuando:

- La aplicación maneja entradas de **varios dispositivos de comunicación.**
- La aplicación debe poder realizar **diferentes tareas** a la vez.
- Interesa **diferenciar tareas** con prioridad variada.
- La aplicación se va a ejecutar en un **entorno multiprocesador.**

Un hilo lleva **asociados**:

- Un **identificador** único.
- Un **contador de programa** propio.
- Un conjunto de **registros**.
- Una pila de **variables locales.**

Y puede **compartir con otros hilos** del proceso:

- Código.
- Datos como **variables globales.**
- Otros **recursos del sistema** operativo.

3. Multihilo en Java. Librerías y clases

3.1. Utilidades de concurrencia del paquete java.lang

- **Thread** Clase responsable de **producir hilos funcionales** para otras clases.
- **Runnable** Interfaz que **añade la funcionalidad de hilo** a las clases que la implementan.
- **ThreadDeath** Deriva de la clase `Error` y proporciona medios para **manejar y notificar errores.**
- **ThreadGroup** Clase que se utiliza **para manejar y controlar la ejecución de un grupo de hilos** de modo conjunto.
- **Object** Clase que proprciona algunos **métodos cruciales** a la arquitectura multihilo.

3.2. Utilidades de concurrencia del paquete java.util.concurrent

- **Clases de sincronización** `Semaphore`, `CountDownLatch`, `CyclicBarrier` y `Exchanger`.
- **Interfaces para separar la lógica de la ejecución** `Executor`, `ExecutorService`, `Callable` y `Future`.
- **java.util.concurrent.atomic** Conjunto de clases para ser usadas como variables atómicas en aplicaciones multihilo.
- **java.util.concurrent.locks** Define una serie de alternativas a la cláusula `synchronized`. (Ver la sección 7.2)

4. Creación de hilos

En Java, un hilo se representa mediante una instancia de la clase `java.lang.thread`, que se puede utilizar para iniciar, detener o cancelar la ejecución.

Se pueden implementar o definir **extendiendo la clase** `Thread` (más sencillo, pero no siempre es posible), o **mediante la interfaz** `Runnable` (procedimiento más general y más flexible). En ambos casos, se debe proporcionar una **definición del método run.**

El siguiente bloque de código muestra un ejemplo de un hilo creado mediante `Thread`:

```
1  public class Saludo extends Thread {
2  //clase que extiende a Thread
3     public void run() {
4     // se redefine el metodo run() con el codigo asociado al hilo
5        System.out.println("Saludo desde un hilo extendiendo thread");
6     }
7     public static void main(String args[]) {
8        Saludo hilo1=new Saludo();
9        //se crea un objeto Thread, el hilo hilo1
10       hilo1.start();
11       //invoca a start() y pone en marcha el hilo hilo1
12    }
13 }
```

Este segundo bloque de código muestra un ejemplo de un hilo creado mediante `Runnable`:

```
1  public class Saludo implements Runnable {
2  //clase que implementa a Runnable
3     public void run() {
4     //se redefine el metodo run()con el codigo asociado al hilo
5        System.out.println("Saludo desde un hilo creado con Runnable");
```

```
6     }
7     public static void main(String args[]) {
8       Saludo miRunnable=new Saludo();
9       //se crea un objeto Saludo
10      Thread hilo1= new Thread(miRunnable);
11      //se crea un objeto Thread (el hilo hilo1) pasando como argumento
12     // al constructor un objeto Saludo
13       hilo1.start();
14      //se invoca al metodo start() del hilo hilo1
15    }
16  }
```

5. Estados de un hilo

Ciclo de vida de un hilo Comprende los diferentes estados en los que puede estar un hilo **desde que se crea hasta que finaliza.**

Los estados son:

- **Nuevo** Aún no está disponible para su ejecución.
- **Ejecutable** El hilo está **preparado para ejecutarse**. Puede estar **ejecutándose**, si se le ha asignado tiempo de procesamiento, o **preparado**, si no está ejecutándose en un instante determinado en **beneficio de otro hilo.**
- **No Ejecutable o Detenido** Podría estar ejecutándose, pero hay alguna actividad **interna al propio hilo** que se lo impide.
- **Muerto o Finalizado** El hilo ha finalizado.

5.1. Iniciar un hilo

El método `thread.start` realiza las siguientes tareas:

- **Crea los recursos** del sistema necesarios para ejecutar el hilo.
- Llama al método `run` y lo ejecuta como un **subproceso nuevo** e independiente.

Consideraciones:

- Se puede **invocar directamente** al método run, pero ejecutará el código **en el hilo principal** en lugar de iniciar un nuevo hilo.

- Una vez se ha llamado al método start de un hilo, **no puede realizarse** otra llamada.
- El orden de ejecución de los hilos es **no-determinístico**.

5.2. Finalizar un hilo

Un hilo muere cuando su método `run` **termina de ejecutarse.** Una vez ha muerto, **no puede ser reiniciado** y debe crearse uno nuevo.

Se puede usar el método `isalive` para comprobar si un hilo está vivo.

6. Gestión y planificación de hilos

Paralelismo En un sistema con múltiples CPU, cada una **ejecuta un hilo diferente**.

Pseudoparalelismo Una CPU es responsable de ejecutar **múltiples hilos**.

El planificador de hilos de Java utiliza un **algoritmo de secuenciación de hilos** basado en un **sistema de prioridades relativas**: secuencia la ejecución de hilos en base a su **prioridad**.

- El hilo elegido para ejecutarse siempre es el **hilo ejecutable de prioridad más alta.**
- Si hay **más de un hilo** con la misma prioridad, el orden de ejecución se maneja mediante un Round Robin con FIFO.
- Cuando un hilo que está ejecutándose pasa a **No Ejecutable o Muerto**, se selecciona otro hilo para su ejecución.
- La ejecución de un hilo se interrumpe si otro hilo con prioridad más alta se vuelve ejecutable. Esto se denomina **planificación apropiativa**.

En los sistemas operativos que **implementan time slicing**, los hilos que entran en ejecución se mantienen en ella solo **durante un quantum.**

En los sistemas operativos que **no implementan time slicing**, los procesos son ejecutados **hasta su muerte.**

6.1. Prioridad de hilos

En Java, cada hilo tiene una prioridad representada por un **valor entre 1 y 10.** Cuanto **mayor el valor, mayor la prioridad.**

El hilo **principal** es creado con **prioridad 5**. Los hilos secundarios **heredan la prioridad** que tenga su hilo padre.

Se puede obtener y modificar la prioridad de un hilo mediante `getPriority` y `setPriority`.

6.2. Hilos egoistas y programación expulsora

Hilo egoista Hilo que, en un sistema operativo **sin time-slicing**, no sale de su ejecución hasta su muerte, **no permitiendo a otros hilos entrar** en ejecución hasta entonces.

El método `yield` en Java obliga que un hilo que **esté ejecutándose pase a estar preparado** para que otros hilos de la misma prioridad puedan ejecutarse.

7. Sincronización y comunicación de hilos

Sincronización Capacidad de **informar de la situación de un hilo a otro** para establecer la **secuencialidad correcta** del programa.

Comunicación Capacidad de **transmitir información desde un hilo a otro** para intercambiar información.

7.1. Información compartida entre hilos

Secciones críticas Aquellas secciones de código que **no pueden ejecutarse concurrentemente** porque contienen **recursos o información compartidos** por diferentes hilos.

7.2. Monitores. Métodos synchronized

Monitor Porción de código protegida por un lock mediante la palabra `synchronized`. Solo **un hilo puede ejecutar esta porción a la vez**: el resto permanecerán bloqueados y en espera.

Puede hacerse a **nivel de método**...

```java
public class ContadorMetodo {
    private int cuenta = 0;

    // El metodo completo esta protegido por el monitor del objeto
    public synchronized void incrementar() {
        cuenta++;
    }

    public int getCuenta() {
        return cuenta;
    }
}
```

. . . o a **nivel de bloque.**

```java
public class ContadorBloque {
    private int cuenta = 0;

    public void incrementar() {
        // Codigo no critico (asincrono) puede ir aqui...

        synchronized (this) {
            // Solo esta seccion critica esta protegida
            cuenta++;
        }

        // Mas codigo no critico aqui...
    }

    public int getCuenta() {
        return cuenta;
    }
}
```

Declarar fragmentos de código como `synchronized` ralentizará la ejecución del programa, ya que la adquisición y liberación de monitores genera una sobrecarga.

7.3. Comunicación entre hilos con métodos de java.lang.Object

- **wait** Detiene el hilo hasta que **otro hilo le notifique** que puede continuar. Acepta un **parámetro opcional de tiempo** antes de que el hilo continúe automáticamente.
- **notify/notifyAll** Notifica a **uno o todos los hilos** del mismo objeto en espera que pueden **continuar ejecutándose.**

7.4. La clase Semaphore

Permite definir un semáforo para **controlar el acceso** a un recurso compartido.

Para crear un semáforo:

- Indicar al constructor `Semaphore(int permisos)` el **número de hilos** que pueden acceder a la vez al recurso.
- Indicar al semáforo, **mediante el método** `acquire`, que queremos acceder al recurso.
- Indicar al semáforo **mediante el método** `release` que libere el recurso.

Si el semáforo es usado para **proteger secciones críticas**, las llamadas a los métodos se harán desde la sección crítica, con un solo hilo.

```
1  import java.util.concurrent.Semaphore;
2
3  public class Ejemplo1 {
4      private static final Semaphore semaforo = new Semaphore(3);
5
6      static class Usuario extends Thread {
7          private String nombre;
8
9          public Usuario(String nombre) {
10             this.nombre = nombre;
11         }
12
13         @Override
14         public void run() {
15             try {
16                 semaforo.acquire();
17                 System.out.println(nombre + " conectado a la base de datos.");
18                 Thread.sleep((long) (Math.random() * 3000));
19                 System.out.println(nombre + " se ha desconectado.");
20
21             } catch (InterruptedException e) {
22                 e.printStackTrace();
23             } finally {
24                 semaforo.release();
25             }
26         }
27     }
28     public static void main(String[] args) {
29         for (int i = 1; i <= 6; i++) {
30             new Usuario("Hilo-" + i).start();
31         }
32     }
33 }
```

Si se usa para **comunicar hilos**, uno usará `acquire` y el otro `release` para trabajar de manera coordinada.

```java
import java.util.concurrent.Semaphore;

public class Ejemplo2 {
    private static final Semaphore semaforoPing = new Semaphore(1);
    private static final Semaphore semaforoPong = new Semaphore(0);

    static class HiloPing extends Thread {
        @Override
        public void run() {
            for (int i = 0; i < 5; i++) {
                try {
                    semaforoPing.acquire();
                    System.out.println("PING");
                    Thread.sleep(500);
                    semaforoPong.release();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        }
    }

    static class HiloPong extends Thread {
        @Override
        public void run() {
            for (int i = 0; i < 5; i++) {
                try {
                    semaforoPong.acquire();
                    System.out.println("PONG");
                    Thread.sleep(500);
                    semaforoPing.release();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        }
    }

    public static void main(String[] args) {
        new HiloPing().start();
        new HiloPong().start();
    }
}
```

7.5. La clase Exchanger

Establece un punto de sincronización donde **se intercambian objetos entre dos hilos.** Se emplea, típicamente, cuando **un hilo productor está rellenando un búfer y otro hilo consumidor lo está consumiendo.**

- El hilo A llama a `exchange(objetoA)` y el hilo B a `exchange(objetoB)`.

- El hilo que procese la llamada en primer lugar se bloqueará y quedará a la espera de que lo haga el segundo, tras lo cual ambos se liberarán.

```
1  import java.util.concurrent.Exchanger;
2
3  public class ExchangerExample {
4      private static final Exchanger<String> exchanger = new Exchanger<>();
5
6      public static void main(String[] args) {
7          new Thread(new Producer()).start();
8          new Thread(new Consumer()).start();
9      }
10
11     static class Producer implements Runnable {
12         @Override
13         public void run() {
14             try {
15                 String data = "Data from Producer";
16                 System.out.println("Producer: " + data);
17                 String response = exchanger.exchange(data);
18                 System.out.println("Producer received: " + response);
19             } catch (InterruptedException e) {
20                 Thread.currentThread().interrupt();
21             }
22         }
23     }
24
25     static class Consumer implements Runnable {
26         @Override
27         public void run() {
28             try {
29                 String data = "Data from Consumer";
30                 System.out.println("Consumer: " + data);
31                 String response = exchanger.exchange(data);
32                 System.out.println("Consumer received: " + response);
33             } catch (InterruptedException e) {
34                 Thread.currentThread().interrupt();
35             }
36         }
37     }
38 }
```

8. La clase CountDownLatch

Utilidad de sincronización que permite que **uno o más hilos esperen hasta que otros hilos terminen su ejecución.**

Implementa un punto de espera donde uno o más hilos permanecen en espera. El objeto controla **cuantos hilos deben completar su trabajo** antes de que el resto puedan continuar.

La clase CyclicBarrier es una variación de esta clase, con la diferencia de que a diferencia de en CountDownLatch, la cuenta atrás de CyclicBarrier se puede reutilizar.

```
1  import java.util.concurrent.CountDownLatch;
2
3  public class CountDownLatchExample {
4      public static void main(String[] args) throws InterruptedException {
5          // Crear un CountDownLatch con un contador de 3
6          CountDownLatch latch = new CountDownLatch(3);
7
8          // Crear tres tareas que se ejecutaran en paralelo
9          Runnable worker = () -> {
10             System.out.println(Thread.currentThread().getName() + " esta trabajando...");
11             try {
12                 Thread.sleep(2000); // Simula trabajo
13                 System.out.println(Thread.currentThread().getName() + " termino su tarea.");
14                 latch.countDown(); // Decrementa el contador
15             } catch (InterruptedException e) {
16                 Thread.currentThread().interrupt();
17             }
18         };
19
20         // Iniciar tres hilos
21         new Thread(worker, "Hilo-1").start();
22         new Thread(worker, "Hilo-2").start();
23         new Thread(worker, "Hilo-3").start();
24
25         // El hilo principal espera hasta que los demas terminen
26         latch.await();
27         System.out.println("Todos los hilos han terminado. Continuando con la ejecucion principal.");
28     }
29 }
```

9. Aplicaciones multihilo

Deben reunir las siguientes propiedades

- **Seguridad** La aplicación no llegará a un estado inconsistente por un mal uso de los recursos compartidos.
- **Viveza** La aplicación no se bloqueará o provocará que un hilo no se pueda ejecutar.

La corrección de la aplicación puede ser

- **Parcial** Se cumple la propiedad de seguridad.
- **Total** Se cumplen las propiedades de seguridad y viveza.

9.1. La interfaz Executory los pools de hilos

Executor Interfaz que permite ejecutar tareas en un **único hilo en segundo plano, en un hilo nuevo, o en un pool de hilos.** También permite diseñar **políticas propias de ejecución** y añadirlas.

Pool de hilos Contenedor dentro del cual se crean e inician un número limitado de hilos para **ejecutar todas las tareas** de una lista.

Un pool puede ser declarado como un objeto `ExecutorService` mediante uno de los métodos de la clase estática `Executor`:

- **newFixedThreadPool** Crea un pool con el número de hilos indicados, que **son reutilizados** hasta terminar con las tareas de la lista.
- **newCachedThreadPool** Crea un pool que va creando hilos **según se van necesitando**, pero puede reutilizar los ya concluidos.
- **newSingleThreadExecutor** Crea un pool de un solo hilo. Si ocurre una excepción durante la ejecución de una tarea, **no detiene la ejecución** de los siguientes.
- **newScheduledExecutor** Crea un pool que ejecuta tareas **programadas** cada cierto tiempo. Similar a un `Timer`, pero puede tener varios hilos que realizarán las tareas programadas según se desocupen.

© 2026 - Esteban N. - estnicolas@proton.me

Este texto se encuentra licenciado bajo Creative Commons Atribución-Reconocimiento/NoComercial-CompartirIgual 4.0 Internacional.
Para ver una copia de esta licencia visite:
https://creativecommons.org/licenses/by-nc-sa/4.0/legalcode.es.



Ninguna parte de este documento, código o texto, ha sido realizada, apoyada, consultada, generada, corregida o involucrada en ninguna forma o contexto con inteligencia artificial, ni lo será en cualquier futura posible versión o corrección.
https://notbyai.fyi